

Technical Design Documents

CHAT SERVER CONSOLE APPLICATION

Evan Allen

Table of Contents

1. Project Scope	2
2. Features & Functionalities	2-3
3. Command Syntax & Format	3-5
4. Server Architecture	5-6
5. File Structure & Project Map	6-7
6. Sequence Diagrams	8-11
7. Error Handling	11-13

1. Project Scope

1.1 Introduction: The CNE Chat Server is a C++ console application engineered to mediate communication between clients. The system establishes a custom application-layer protocol to interface directly with the provided SpaghettiRelay client program. To guarantee reliable data exchange, all network payloads are explicitly framed utilizing a single-byte length prefix followed immediately by the message content.

1.2 Purpose: The primary objective is to deploy a stable, event-driven chat system with comprehensive multi-client support. The server architecture is built upon the following core systems:

- **Transport Protocol:** The application relies exclusively on TCP for client-server communication. This guarantees in-order packet delivery and prevents message dropping, which is critical for accurate chat relay and logging.
- **Concurrency Model:** To bypass the performance bottleneck of blocking network I/O, the server implements in-thread multiplexing via the `select()` system call. This event-driven approach allows a single primary execution thread to manage multiple active socket descriptors simultaneously without halting operations.
- **Service Discovery:** A distinct secondary thread is allocated specifically for a UDP socket. This thread is responsible for constructing and dispatching a broadcast message containing the server's connection parameters at regular intervals to the local network.

2. Features & Functionalities

The server engine requires the implementation of several core networked systems to facilitate the chat environment and manage active client sockets:

- **User Registration & Authentication:** Validates user access through account creation and login requests. To ensure constant-time average-case

complexity for credential verification and insertion, all user data is managed in memory using a custom Hashtable data structure.

- **Session Management & Tracking:** Maintains the state of active socket connections, enforcing a strict single-session rule that prevents users from registering or logging in if they are already active. The server supports dynamic retrieval of this active client pool via user request.
- **Message Routing (Relay & Direct):** The default server behavior iterates through the active client list to broadcast standard message payloads to all connected sockets. This loop is bypassed when a user requests one-on-one direct messaging, allowing targeted payload delivery to a specific client.
- **Dual-Stream Logging:** The system persistently records server activity by generating two distinct log files on disk. System commands are routed to one file, while public message traffic is routed to the other, allowing users to safely retrieve the public chat history upon request.
- **Graceful Connection Termination:** Handles explicit client disconnects by gracefully closing the socket, initiating the standard TCP termination handshake (FIN/ACK/FIN/ACK), and purging the user from the active session pool to free up chat capacity.
- **UDP Service Broadcasting:** Operates on an isolated `std::thread` to construct and dispatch a UDP datagram. This broadcast utilizes the `sendto()` function at regular intervals to announce the server's IP address and Port across the local subnet.

3. Command Syntax & Format

3.1 Command Protocol & Prefix: The server parses all incoming application-layer frames to determine routing logic. The operational command prefix for this deployment is the tilde (~). Any payload originating from an authenticated client that does not begin with this prefix is automatically classified as a standard public chat broadcast.

3.2 Command Definitions & Expected Logic:

Below is the strict syntax, parameter requirement, and expected server-side logic for the supported command set.

- **Help Utility:** ~help
 - Parameters: None
 - Expected Response: Transmits a formatted string containing the complete list of available server commands and their operational syntax back to the requesting client.
 - Edge Cases: Safely dropping or ignoring any extraneous arguments appended after the base command.
- **User Registration:** ~register <username> <password>
 - Parameters: username (string), password (string)
 - Expected Response: The server executes an insertion into the credential Hashtable. It returns a definitive status indicator: Success, Failure, or Decline. Note that a successful registration does not automatically authenticate the socket.
 - Edge Cases: Registration is explicitly declined if a username collision occurs within the Hashtable, or if the server's predefined chat capacity limit has been reached.
- **Client Authentication:** ~login <username> <password>
 - Parameters: username (string), password (string)
 - Expected Response: The server queries the Hashtable for the provided key. It returns a status of Success, User Not Found, or Incorrect Password.
 - Edge Cases: The server actively rejects the login attempt if the socket is already authenticated, or if the targeted user profile is currently active on another session, strictly preventing concurrent logins.
- **Direct Messaging:** ~send <recipient_username> <message_text>
 - Parameters: recipient_username (string), message_text (string)
 - Expected Response: Intercepts the payload and overrides the default public relay logic, routing the message_text exclusively to the active socket mapped to recipient_username.
 - Edge Cases: The server returns an error notification to the sender if the requested recipient does not exist or is currently offline.

- **Active Client Retrieval:** ~getlist
 - Parameters: None
 - Expected Response: Compiles the current roster of authenticated, active clients and transmits the data stream to the requesting socket.
 - Edge Cases: Returns an empty list or specific system message if the requesting user is the only active client currently on the server.
- **Public Log Retrieval:** ~getlog
 - Parameters: None
 - Expected Response: The server reads the designated public message log file from the disk and streams the text content directly to the requesting client.
 - Edge Cases: Safely handling OS-level file read locks or returning a predefined indicator if the log file is currently empty.
- **Session Termination:** ~logout
 - Parameters: None
 - Expected Response: Purges the user from the active session pool, preventing further message relays, and initiates a graceful TCP disconnection handshake (FIN/ACK/FIN/ACK).
 - Edge Cases: Ensuring the server does not crash if an abrupt socket closure (e.g., client network drop or forced exit) occurs before the logout command is successfully parsed.

4. Server Architecture

4.1 Architectural Overview: The server engine is built upon an Event-Driven Architecture utilizing blocking I/O. To manage multiple client socket descriptors simultaneously without halting the primary execution thread, the system implements in-thread multiplexing via the `select()` system call. This approach allows the server to efficiently monitor a dynamic `fd_set` of connected sockets, only allocating CPU cycles to process I/O operations when `select()` flags a specific socket as ready for reading, writing, or exception handling.

4.2 System Components: The application logic is modularized into four primary operational components:

- **ChatServer:** Operates as the core system orchestrator. It is responsible for initializing the master listening socket, binding it to the designated port, and driving the primary select() execution loop to handle all incoming client connections and state changes.
- **ClientHandler:** Manages the lifecycle and state data of individual client interactions. This includes maintaining socket identifiers, tracking authentication status, and wrapping send/receive logic for isolated client streams.
- **MessageHandler:** Acts as the central parsing and routing engine. It intercepts raw payloads from the ChatServer, decodes the application-layer framing, executes command logic (identifying the ~ prefix), and routes outgoing payloads to their appropriate target sockets.
- **Logger:** An isolated file I/O component responsible for generating and appending data to persistent .txt files on disk. It safely records all public message traffic and internal server events (such as parsed commands) into their respective logs.

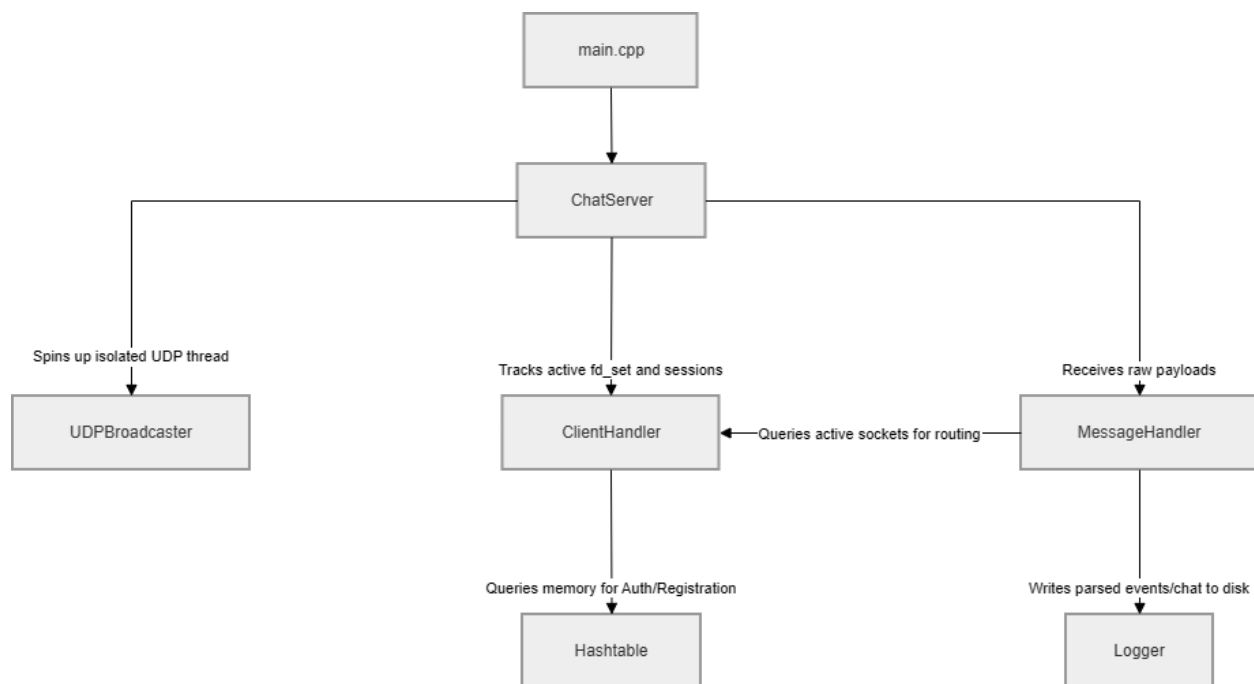
5. File Structure & Project Map

5.1 Code Organization: The project repository is strictly modularized to separate networking logic, state management, and file I/O. Below is the foundational file structure mapping the header (.h) interfaces to their respective source (.cpp) implementations:

- **main.cpp:** System entry point; handles initial configuration and spins up the server engine.
- **ChatServer.h / .cpp:** Encapsulates the master select() loop, TCP socket initialization, and inbound connection routing.
- **ClientHandler.h / .cpp:** Manages the active session pool, socket descriptors, and user authentication states.
- **MessageHandler.h / .cpp:** The routing engine; handles payload decoding, command prefix (~) execution, and outbound data streams.

- **Hashtable.h / .cpp:** Custom memory-optimized data structure for constant-time credential storage and lookup.
- **Logger.h / .cpp:** Dedicated File I/O module managing asynchronous writes to the public chat and system command .txt files.
- **UDPBroadcaster.h / .cpp:** Houses the isolated std::thread logic for the recurring UDP service discovery datagrams.

5.2 Dependency Map



6. Sequence Diagrams

Figure 6.1 -User Registration

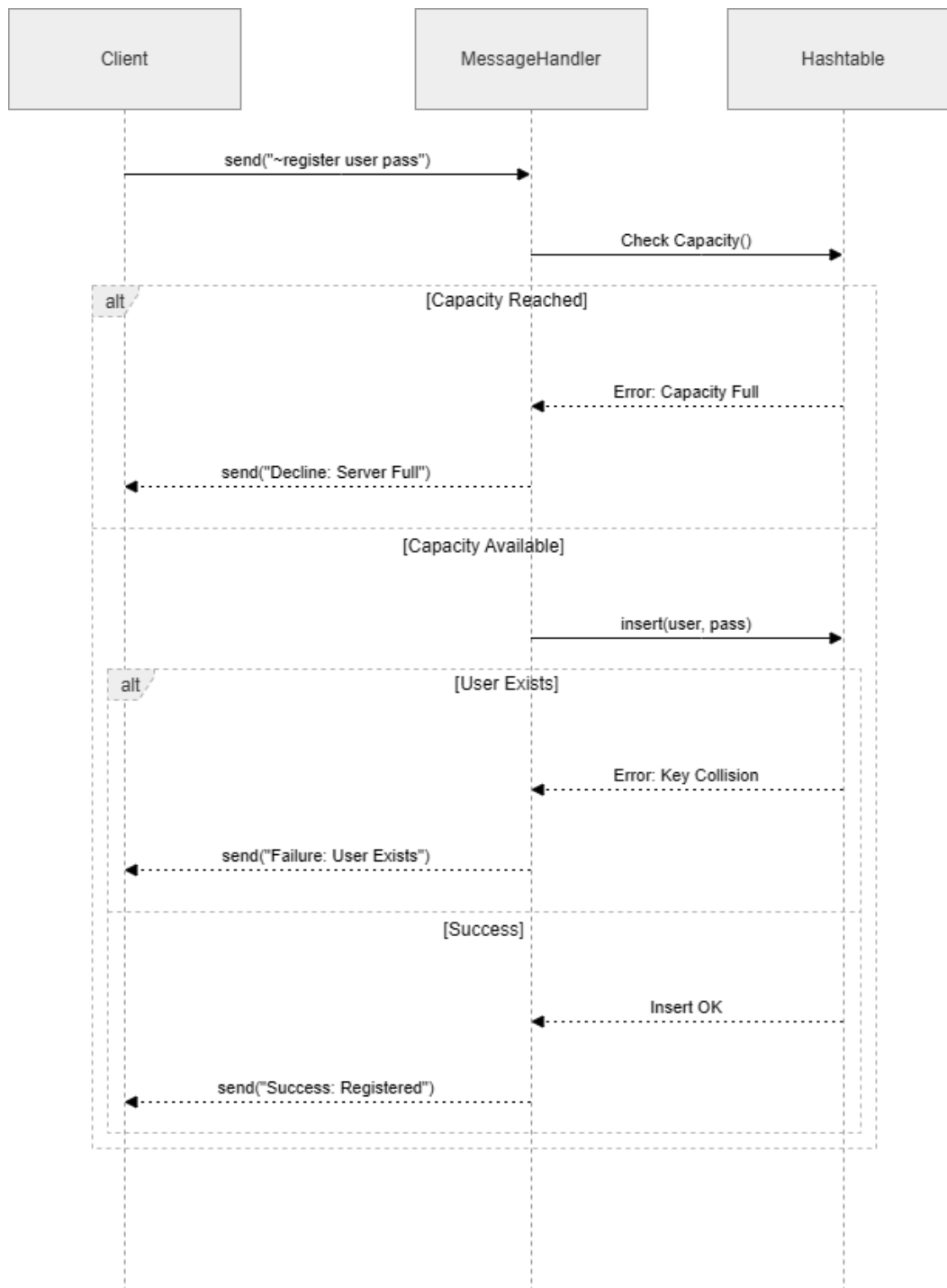


Figure 6.2 – Login Process

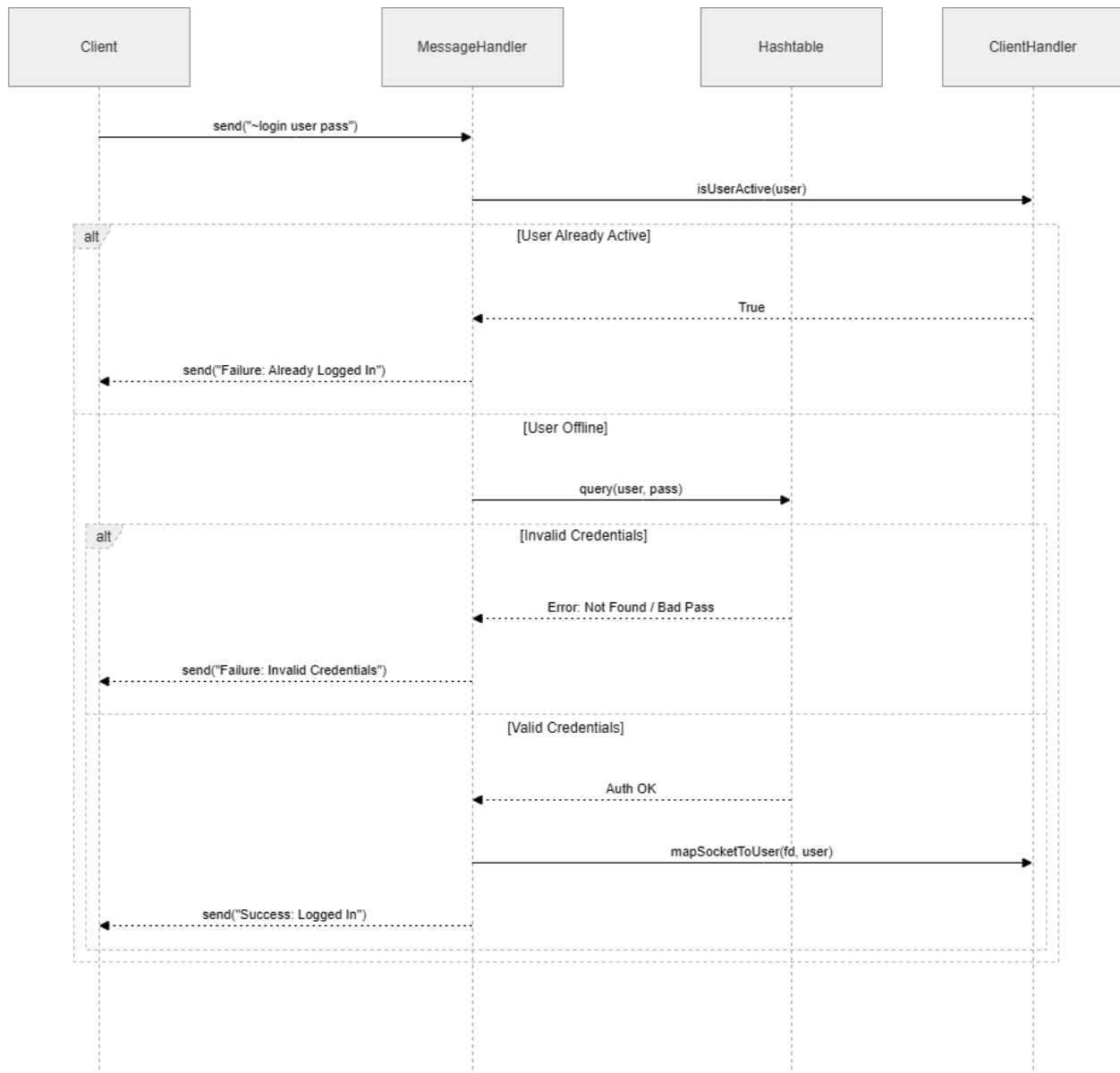


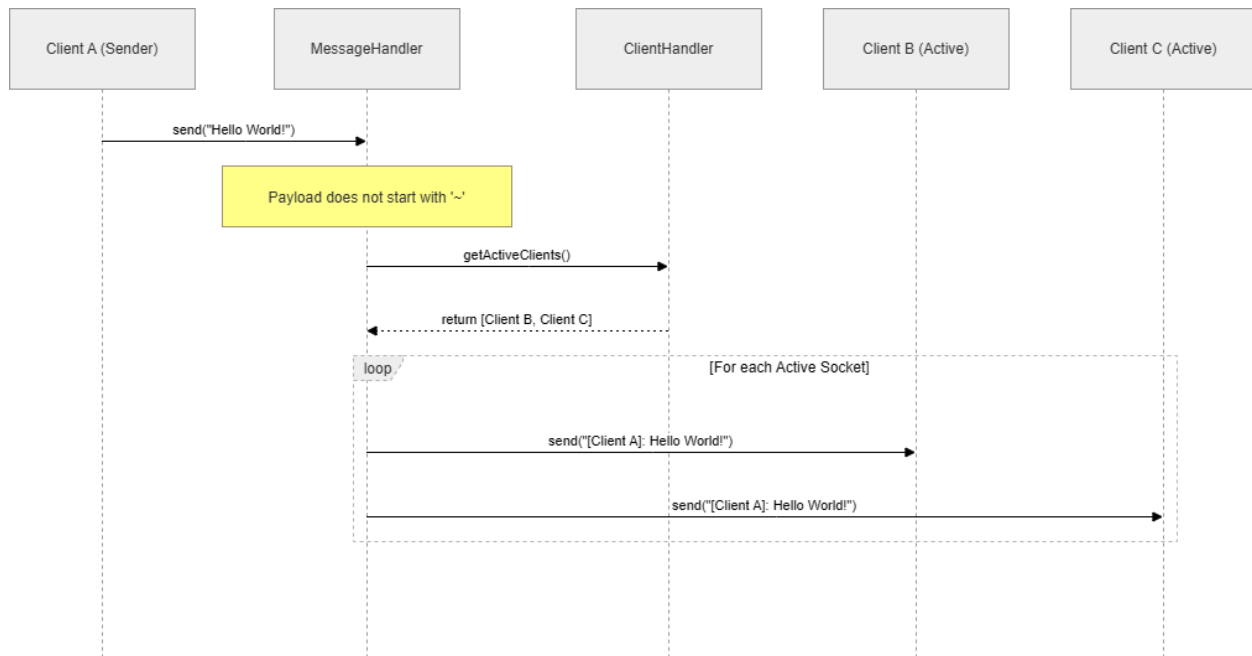
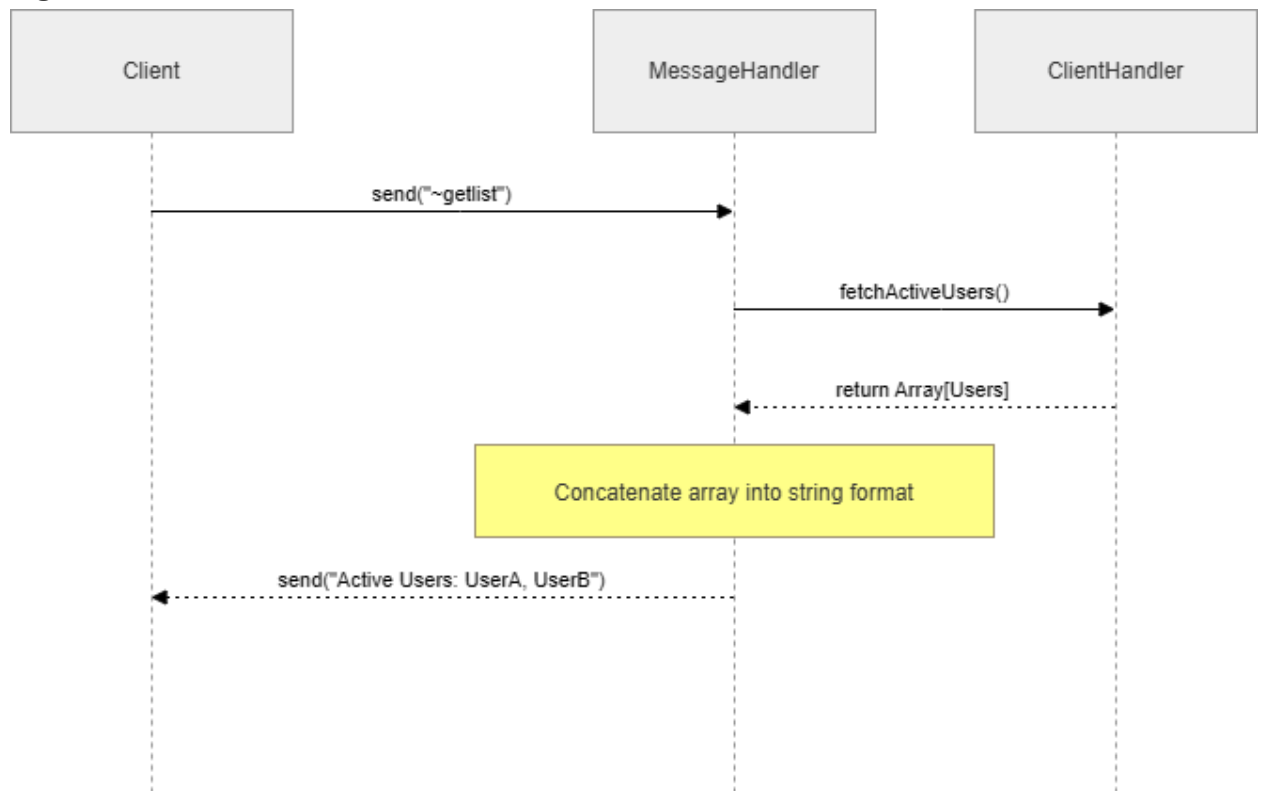
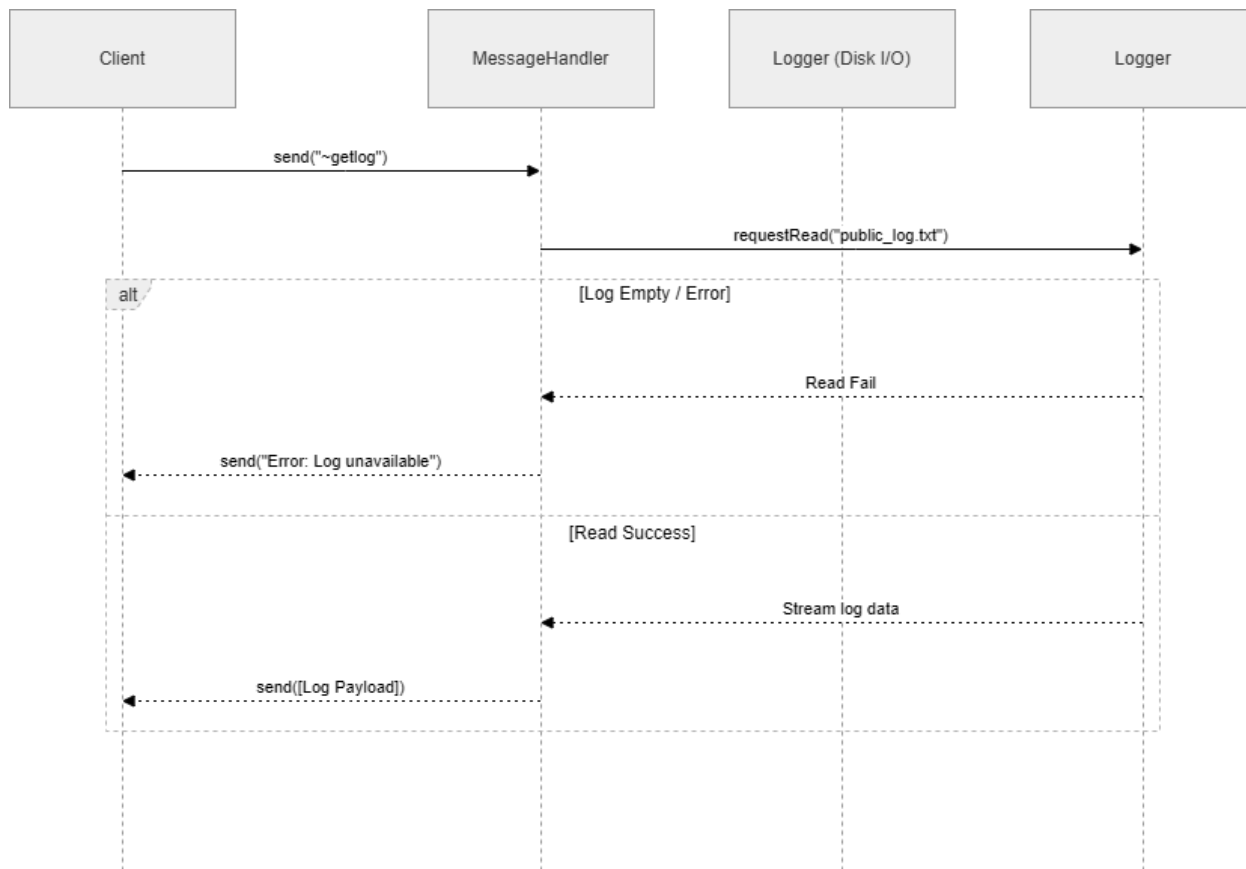
Figure 6.3 – Public Messaging Flow**Figure 6.4 – User List Retrieval**

Figure 6.5 Log Retrieval

7. Error Handling

7.1 System Resilience & Mitigation Strategies: The server engine incorporates strict validation and error-trapping mechanisms to maintain stability during anomalous network events, hardware faults, or malformed inputs.

- **Socket Initialization Errors (Creation, Binding, Listening, Accepting):**
 - *Scenario:* Failure to acquire OS resources or bind to an occupied network port.
 - *Mitigation:* The server validates the return value of all socket API calls against `SOCKET_ERROR`. Fatal errors during startup trigger an immediate console output of the specific system error code, followed by a safe system abort. The `SO_REUSEADDR` socket option is actively utilized to mitigate port-lock issues after a server restart.

- **Abrupt Connection Drops:**

- *Scenario:* A client loses network connectivity or force-quits the application without issuing the ~logout command.
- *Mitigation:* When `recv()` returns 0 (graceful closure) or a negative value (error), the ChatServer immediately isolates the faulting socket. It purges the descriptor from the active `fd_set`, closes the socket, and removes the associated user from the active session pool to prevent ghost connections.

- **Authentication Failures:**

- *Scenario:* A client provides an incorrect password or non-existent username during the ~login sequence.
- *Mitigation:* The Hashtable query returns null or false. The server explicitly denies authentication, transmits an "Incorrect Password" or "User Not Found" payload back to the client, and leaves the socket open but restricted in an unauthenticated state.

- **Invalid Command Format:**

- *Scenario:* A client transmits a command with missing or malformed parameters (e.g., sending ~register admin without a password).
- *Mitigation:* The MessageHandler performs strict bounds checking on the parsed payload array. If parameter counts do not match the required syntax, the server aborts execution and returns a formatted syntax error to the client.

- **Unknown Command Input:**

- *Scenario:* A payload begins with the command prefix (~), but the subsequent string does not match any predefined system operations.
- *Mitigation:* The command routing switch drops into a default fallback state. The server transmits an "Unrecognized Command" notification to the client and automatically prompts them to execute the ~help command.

- **Duplicate Login Attempts:**

- Scenario: An authenticated user attempts to log in from a second socket, or an unauthenticated socket attempts to log into a profile that is currently active elsewhere.
 - Mitigation: The ClientHandler queries the active session pool *prior* to querying the Hashtable. If a session match is found, the server intercepts the request, returns a "User Already Logged In" error, and preserves the integrity of the original active session.
- **Invalid Server Admin Input:**
 - Scenario: The administrator provides out-of-bounds parameters (Port, Chat Capacity, Command Character) during the initial server startup configuration.
 - Mitigation: The initialization sequence utilizes an input validation loop. It strictly enforces type checking (e.g., verifying capacity is an integer) and bounds checking before allowing the master ChatServer engine to initialize.